

Partial gamma DIF null with unequal groups

Does random group assignment in `RMdifGammaCutoff()` hold the error rate when groups differ in θ ?

Magnus Johansson

2026-10-03

Table of contents

Purpose	1
Four nulls	1
Design	1
Results	5
Findings	7

Purpose

`RMdifGammaCutoff()` builds its null by resampling WLE estimates from the pooled sample and assigning simulated respondents to groups **at random**, with the observed proportions. The simulated groups therefore always share one θ distribution. Real groups often differ in mean θ , which is the situation where DIF analysis matters most.

Under the Rasch model the item response is independent of group given the total score, so the null hypothesis holds whatever the groups' θ distributions. But the null **distribution** of partial gamma depends on how each group is spread across score strata. With a large group difference, the extreme strata hold mostly one group and contribute few comparable pairs. A null built from equal groups may then have the wrong spread. [dev/dif-impact-design.md](#), Part A, question 6, leaves the choice of null to this study.

Four nulls

1. **random**: the current method. Pooled WLE pool, groups assigned at random with multinomial sizes, CML thresholds fixed, iterations with an empty category (polytomous) or fewer than 8 positive responses (dichotomous) dropped, as in `run_single_partgam_sim()`.
2. **groupwise**: each group resampled from its own WLE estimates under the pooled CML thresholds, group sizes fixed at the observed sizes, the same validity check.
3. **conditional**: every respondent keeps their observed group and total score, and a new response pattern is drawn from the Rasch conditional distribution given that score (`.sim_cond_dataset()`, the package's conditional DGP).
4. **permutation**: group labels permuted within strata of the total score. Given the score, the Rasch model makes response patterns exchangeable across groups, so this null is exact and needs no item parameters. The same permutation is applied to all items, which keeps their dependence for the Westfall-Young step-down.

Nulls 3 and 4 condition on the observed (group, score) pairs, so in theory they are unaffected by the group difference. Null 2 reproduces the group difference without conditioning. Null 1 ignores it.

Design

- **Items**. `poly9`: 9 items with 4 categories, locations -1.5 to 1.5 , thresholds at location ± 0.8 and the location. `dich15`: 15 dichotomous items, locations -2 to 2 .

- **Persons.** Group 1 $\theta \sim N(0, 1.5^2)$, group 2 $\theta \sim N(\Delta \cdot 1.5, 1.5^2)$, with $\Delta \in \{0, 0.5, 1\}$ SD.
- **Group sizes.** 150/150 and 480/120.
- **No DIF.** Every flag is a false positive.
- **Replications.** $R = 300$ datasets per condition, $B = 400$ iterations per null (the approved new default).
- **Statistic.** The vectorised partial gamma (`dif_partgam_fast.R`), identical to `iarm::partgam_DIF()`. p-values from the package's `.bootstrap_pvalues()`, two-sided, as `RMdifGamma()` computes them.
- **Outcomes.**
 - Per-item marginal rejection at $\alpha = .05$, pooled over items and datasets.
 - Family-wise rate: share of datasets with any Westfall-Young adjusted $p < .05$.
 - Spread: each null's SD of gamma per item, divided by the true sampling SD of gamma under the generating model (2000 datasets per condition), median over datasets and averaged over items. A ratio below 1 means the null is too narrow and the test anti-conservative.

The Monte Carlo SE of a family-wise rate near .05 at $R = 300$ is about .013. The pooled per-item rate rests on 300×9 or 300×15 tests, but the items within a dataset are dependent, so its effective precision is lower than that count suggests.

```
suppressMessages(devtools::load_all("../", quiet = TRUE))
source("dif_partgam_fast.R")
library(parallel)

out_dir <- "dif_gamma_null_groups"
dir.create(out_dir, showWarnings = FALSE)
R_reps <- 300L
B_iter <- 400L
n_true <- 2000L
cores <- 10L

item_sets <- list(
  poly9 = lapply(seq(-1.5, 1.5, length.out = 9), function(l) l + c(-0.8, 0, 0.8)),
  dich15 = as.list(seq(-2, 2, length.out = 15))
)
for (nm in names(item_sets)) {
  names(item_sets[[nm]]) <- paste0("I", seq_along(item_sets[[nm]]))
}
grid <- expand.grid(
  items = names(item_sets),
  delta = c(0, 0.5, 1),
  sizes = c("150/150", "480/120"),
  stringsAsFactors = FALSE
)
grid$id <- sprintf("%s_d%s_s%s", grid$items, grid$delta, sub("/", "-", grid$sizes))

# Vectorised PCM simulation (distributionally identical to the package's
# sim_partial_score() and psychotools::rrm(), much faster).
sim_pcm_fast <- function(theta, thr_list) {
  n <- length(theta)
  vapply(thr_list, function(thr) {
    m <- length(thr)
    cum <- outer(theta, 0:m) - matrix(c(0, cumsum(thr)), n, m + 1L, byrow = TRUE)
    p <- exp(cum - apply(cum, 1, max))
    cp <- t(apply(p / rowSums(p), 1, cumsum))
    as.integer(rowSums(stats::runif(n) > cp[, -(m + 1L), drop = FALSE]))
  }, integer(n))
}
```

```

}

valid_sim <- function(X, n_cats) {
  if (all(n_cats == 2L)) return(all(colSums(X) >= 8L))
  all(vapply(seq_len(ncol(X)), function(j) all(tabulate(X[, j] + 1L, nbins =
n_cats[j]) > 0L), logical(1)))
}

gen_observed <- function(cond) {
  n <- as.integer(strsplit(cond$sizes, "/")[[1]])
  thr <- item_sets[[cond$items]]
  theta <- c(stats::rnorm(n[1], 0, 1.5), stats::rnorm(n[2], cond$delta * 1.5, 1.5))
  g <- factor(rep(c("g1", "g2"), n))
  X <- sim_pcm_fast(theta, thr)
  colnames(X) <- names(thr)
  list(X = X, g = g)
}

gam <- function(X, g) {
  out <- partgam_dif_fast(X, g)
  stats::setNames(out$gamma, out$item)
}

```

```

# One dataset: observed gamma plus B draws from each null.
null_draws <- function(obs, B) {
  X <- obs$X
  g <- obs$g
  n <- nrow(X)
  n_cats <- apply(X, 2, max) + 1L
  pool <- .wle_theta_pool(X)
  thr <- pool$thr_list
  wle <- .estimate_thetas(X, thr, method = "WLE")$theta
  props <- as.numeric(table(g)) / n
  lev <- levels(g)
  wle_g <- lapply(lev, function(l) { w <- wle[g == l]; w[is.finite(w)] })
  n_g <- as.integer(table(g))
  cond_dl <- list(thr_list = thr, cond_groups = .cond_groups(X, thr),
                 sample_n = n, n_items = ncol(X))
  score <- rowSums(X)
  strata <- split(seq_len(n), score)

  draw <- function(method) {
    repeat {
      if (method == "random") {
        th <- sample(pool$thetas, n, replace = TRUE)
        Xs <- sim_pcm_fast(th, thr)
        gs <- factor(sample(lev, n, replace = TRUE, prob = props), levels = lev)
        if (valid_sim(Xs, n_cats)) break
      } else if (method == "groupwise") {
        th <- unlist(lapply(seq_along(lev), function(k) sample(wle_g[[k]], n_g[k],
replace = TRUE))))
        Xs <- sim_pcm_fast(th, thr)
        gs <- factor(rep(lev, n_g), levels = lev)

```

```

    if (valid_sim(Xs, n_cats)) break
  } else if (method == "conditional") {
    Xs <- as.matrix(.sim_cond_dataset(cond_dl))
    gs <- g
    break
  } else {
    Xs <- X
    gs <- g
    for (s in strata) if (length(s) > 1L) gs[s] <- gs[s[sample.int(length(s))]]
    break
  }
}
colnames(Xs) <- colnames(X)
gam(Xs, gs)
}
methods <- c("random", "groupwise", "conditional", "permutation")
sims <- lapply(stats::setNames(methods, methods), function(mt) {
  do.call(rbind, lapply(seq_len(B), function(b) draw(mt)))
})
list(observed = gam(X, g), sims = sims)
}

summarise_dataset <- function(nd) {
  do.call(rbind, lapply(names(nd$sims), function(mt) {
    pv <- .bootstrap_pvalues(nd$observed, nd$sims[[mt]], correction = "fwer", tail
= "two.sided")
    data.frame(
      method = mt,
      item = pv$name,
      p = pv$p,
      padj = pv$padj,
      null_sd = apply(nd$sims[[mt]], 2, stats::sd)[pv$name],
      stringsAsFactors = FALSE
    )
  })))
}

```

```

RNGkind("L'Ecuyer-CMRG")
for (i in seq_len(nrow(grid))) {
  cond <- grid[i, ]
  f <- file.path(out_dir, paste0(cond$id, ".rds"))
  if (file.exists(f)) next
  t0 <- Sys.time()
  set.seed(1000L + i)
  true_g <- do.call(rbind, mclapply(seq_len(n_true), function(r) {
    o <- gen_observed(cond)
    gam(o$X, o$g)
  }, mc.cores = cores))
  true_sd <- apply(true_g, 2, stats::sd)
  res <- mclapply(seq_len(R_reps), function(r) {
    o <- gen_observed(cond)
    out <- tryCatch(summarise_dataset(null_draws(o, B_iter)), error = function(e)
NULL)
  })
}

```

```

    if (!is.null(out)) out$rep <- r
    out
  }, mc.cores = cores)
res <- do.call(rbind, res)
res$true_sd <- true_sd[res$item]
res <- cbind(cond[rep(1, nrow(res)), c("items", "delta", "sizes")], res)
attr(res, "runtime") <- Sys.time() - t0
saveRDS(res, f)
}

```

Results

```

all_res <- do.call(rbind, lapply(file.path(out_dir, paste0(grid$id, ".rds")),
readRDS))
key <- c("items", "sizes", "delta", "method")
summ <- do.call(rbind, lapply(split(all_res, all_res[key], drop = TRUE), function(x) {
  fw <- tapply(x$padj, x$rep, function(p) any(p < 0.05, na.rm = TRUE))
  sd_ratio <- tapply(x$null_sd / x$true_sd, x$item, stats::median, na.rm = TRUE)
  data.frame(
    items = x$items[1], sizes = x$sizes[1], delta = x$delta[1], method = x$method[1],
    per_item = round(mean(x$p < 0.05, na.rm = TRUE), 3),
    familywise = round(mean(fw), 3),
    sd_ratio = round(mean(sd_ratio), 3),
    reps = length(fw)
  )
}))
summ$method <- factor(summ$method, c("random", "groupwise", "conditional",
"permutation"))
summ <- summ[order(summ$items, summ$sizes, summ$delta, summ$method), ]
rownames(summ) <- NULL
summ

```

items	sizes	delta	method	per_item	familywise	sd_ratio	reps
dich15	150/150	0.0	random	0.046	0.043	1.016	300
dich15	150/150	0.0	groupwise	0.046	0.043	1.016	300
dich15	150/150	0.0	conditional	0.048	0.047	1.002	300
dich15	150/150	0.0	permutation	0.050	0.067	0.996	300
dich15	150/150	0.5	random	0.050	0.063	0.999	300
dich15	150/150	0.5	groupwise	0.048	0.057	1.012	300
dich15	150/150	0.5	conditional	0.048	0.060	1.001	300
dich15	150/150	0.5	permutation	0.050	0.063	0.995	300
dich15	150/150	1.0	random	0.063	0.060	0.954	300
dich15	150/150	1.0	groupwise	0.047	0.023	1.007	300
dich15	150/150	1.0	conditional	0.051	0.033	1.006	300
dich15	150/150	1.0	permutation	0.048	0.030	0.999	300
dich15	480/120	0.0	random	0.050	0.043	1.007	300
dich15	480/120	0.0	groupwise	0.048	0.057	1.005	300

items	sizes	delta	method	per_item	familywise	sd_ratio	reps
dich15	480/120	0.0	conditional	0.053	0.057	0.993	300
dich15	480/120	0.0	permutation	0.052	0.060	0.990	300
dich15	480/120	0.5	random	0.052	0.053	0.986	300
dich15	480/120	0.5	groupwise	0.046	0.043	1.004	300
dich15	480/120	0.5	conditional	0.048	0.037	0.997	300
dich15	480/120	0.5	permutation	0.052	0.040	0.993	300
dich15	480/120	1.0	random	0.075	0.137	0.916	300
dich15	480/120	1.0	groupwise	0.046	0.047	0.999	300
dich15	480/120	1.0	conditional	0.044	0.043	1.004	300
dich15	480/120	1.0	permutation	0.050	0.050	1.001	300
poly9	150/150	0.0	random	0.049	0.057	1.017	300
poly9	150/150	0.0	groupwise	0.050	0.063	1.020	300
poly9	150/150	0.0	conditional	0.049	0.063	1.009	300
poly9	150/150	0.0	permutation	0.054	0.060	1.005	300
poly9	150/150	0.5	random	0.046	0.030	0.993	300
poly9	150/150	0.5	groupwise	0.040	0.027	1.009	300
poly9	150/150	0.5	conditional	0.045	0.030	0.998	300
poly9	150/150	0.5	permutation	0.043	0.037	0.993	300
poly9	150/150	1.0	random	0.059	0.073	0.951	300
poly9	150/150	1.0	groupwise	0.047	0.053	1.010	300
poly9	150/150	1.0	conditional	0.047	0.067	1.014	300
poly9	150/150	1.0	permutation	0.050	0.053	1.007	300
poly9	480/120	0.0	random	0.050	0.047	1.005	300
poly9	480/120	0.0	groupwise	0.047	0.060	1.008	300
poly9	480/120	0.0	conditional	0.056	0.057	0.998	300
poly9	480/120	0.0	permutation	0.052	0.057	0.995	300
poly9	480/120	0.5	random	0.051	0.060	0.983	300
poly9	480/120	0.5	groupwise	0.046	0.037	1.002	300
poly9	480/120	0.5	conditional	0.048	0.060	0.997	300
poly9	480/120	0.5	permutation	0.049	0.040	0.993	300
poly9	480/120	1.0	random	0.089	0.147	0.912	300
poly9	480/120	1.0	groupwise	0.058	0.057	0.999	300
poly9	480/120	1.0	conditional	0.055	0.047	1.003	300
poly9	480/120	1.0	permutation	0.057	0.047	0.999	300

```
library(ggplot2)
mcse <- sqrt(0.05 * 0.95 / R_reps)
ggplot(summ, aes(delta, familywise, colour = method)) +
```

```

  annotate("rect", xmin = -Inf, xmax = Inf, ymin = 0.05 - 2 * mcse, ymax = 0.05 + 2
* mcse,
        alpha = 0.15) +
  geom_hline(yintercept = 0.05, linetype = "dashed") +
  geom_line() +
  geom_point() +
  facet_grid(items ~ sizes) +
  labs(x = "Group difference in  $\theta$  (SD units)", y = "Family-wise rate", colour =
"Null") +
  theme_minimal()

```

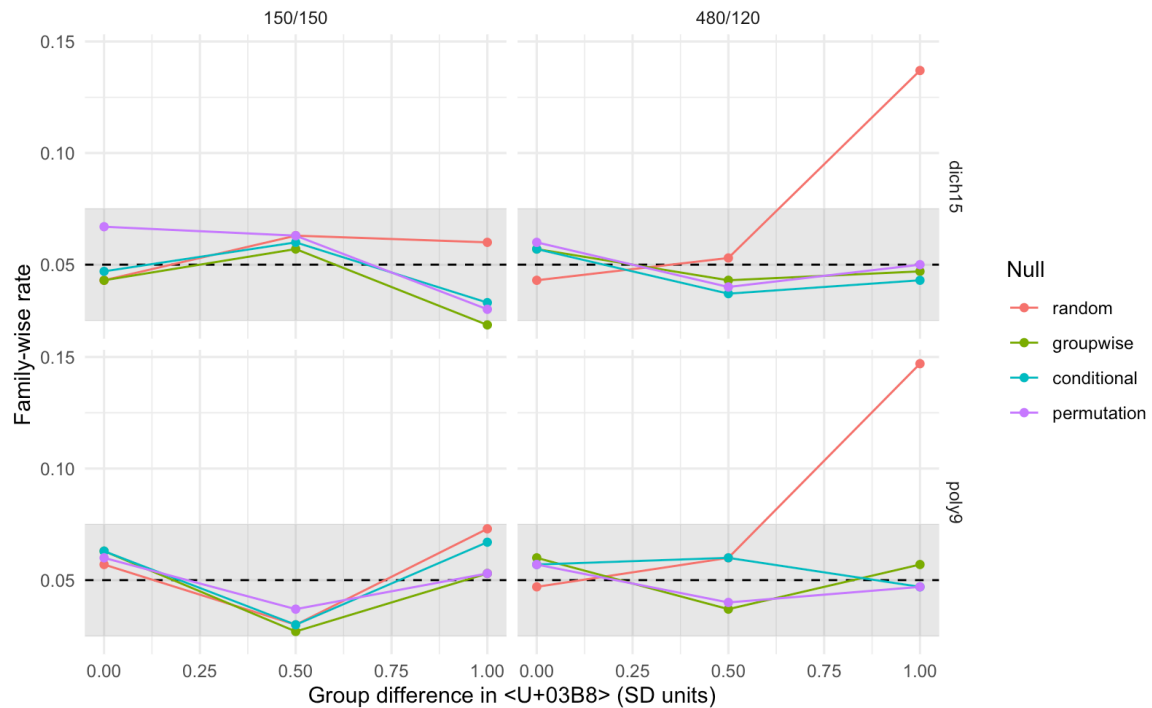


Figure 1: Family-wise false-positive rate (Westfall-Young, $\alpha = .05$) by group difference, null and design. Dashed line at .05, band ± 2 Monte Carlo SE.

```

vapply(file.path(out_dir, paste0(grid$id, ".rds")),
  function(f) as.numeric(attr(readRDS(f), "runtime"), units = "mins"), numeric(1))
|>
stats::setNames(grid$id) |>
round(1)

```

poly9_d0_150-150	dich15_d0_150-150	poly9_d0.5_150-150	dich15_d0.5_150-150
7.3	7.6	7.1	7.4
poly9_d1_150-150	dich15_d1_150-150	poly9_d0_480-120	dich15_d0_480-120
7.0	7.3	11.8	13.6
poly9_d0.5_480-120	dich15_d0.5_480-120	poly9_d1_480-120	dich15_d1_480-120
11.7	13.7	11.8	13.5

Findings

Run 2026-10-03, about two hours on 10 cores. The Monte Carlo SE of a family-wise rate near .05 at $R = 300$ is .013, so rates between about .025 and .075 are compatible with nominal.

1. **The current null is miscalibrated when the groups differ by 1 SD and are unequal in size.** With 480/120 and $\Delta = 1$, the random-assignment null gives a family-wise rate of .137 (dichotomous) and .147 (polytomous), about three times nominal, and per-item rates of .075 and .089. With equal groups at $\Delta = 1$ the excess is smaller (per item .063 and .059, family-wise .060 and .073). At $\Delta \leq 0.5$ it is within Monte Carlo error.
2. **The mechanism is a null that is too narrow, and it is item-specific.** At 480/120 and $\Delta = 1$ the random null's SD is 0.75 to 0.77 of the true sampling SD for the easiest item and rises towards 1 for items near the centre (I1 is the easiest item in both sets). The likely reason is the group composition of the score strata: with a 1 SD difference, the strata that carry information about an easy item hold mostly one group, so fewer comparable pairs exist than in a null with equal groups. This explanation is not tested directly here. The cutoff band from `RMdifGammaCutoff()` is affected in the same way, since it is built from the same simulated values.
3. **All three alternatives hold the error rate in every condition.** Family-wise rates are .023 to .063 for groupwise, .030 to .067 for conditional and .030 to .067 for permutation, and per-item rates .040 to .058. Their null SDs are within 2% of the true SD on average and never below 0.958 for any item.
4. **Choosing among them.** The three are equally well calibrated here, so the choice rests on other grounds:
 - *Permutation* within score strata is exact under the Rasch model, needs no item parameters or person estimates, never produces an invalid simulated dataset, and is the cheapest by far (no data generation). It is a direct test of item $\perp$ group | score, which is the hypothesis partial gamma measures. It also gives a no-DIF reference for the impact function (Part B), since it removes only the item-group association.
 - *Conditional* is the package's existing `dgp = "conditional"`, already the default in `RMitemRestscoreCutoff()`, so it would be the more consistent choice across functions.
 - *Groupwise* is the smallest change to the current code but is the only one of the three that depends on WLE pools and is not exact.

Recommendation for Part A question 6: replace random assignment. Permutation as the default, with the conditional DGP as the alternative if consistency with the restscore cutoff is preferred. Random assignment should not remain an option, since it fails in a common design (a minority group with a different mean).